

# Part 1: Data Engineering Foundations

Comprehensive Overview & Key Architecture Concepts

## Chapter 1: Data Engineering Described

*In this chapter we define what is data engineering and what is its main roles and skills with the right comparison with another data roles*

### 1. What is Data Engineering?

**A.** It is the process of building, developing and maintaining a system that collect, store, ingest, transform and serve the data to the right users and stakeholders (ML engineer, DS, AI engineer, DA, DL engineer, etc.)

### 2. Evolution of the Data Engineer?

- **1980 to 2000:** Data warehousing has been introduced in 1989 by Bill Inmon and Ralph Kimball which made large processing of data possible and accessible and this helped the Internet in 1990 to take a strong leap forward.
- **In early 2000:** The strong start of the internet and the huge amount of data started to accumulate with those traditional monolithic relational databases, and data warehouse problems started to appear to handle this huge amount of data in the back-end which lead to the start of the big data era and the innovations on handling it with google researches and yahoo creating Hadoop and amazon creating AWS which give a lot of services to handle big data and started the cloud.
- **2000 to 2010s:** Big data engineering started with Hadoop then the open source softwares started to rise and there were Apache Pig, Apache Hive, Dremel, Apache HBase, Apache Storm, Apache Cassandra, Apache Spark, Presto all of technologies to handle the big data but those systems were too hard to maintain and develop by companies so engineers tried to simplify and make the process abstract as possible
- **2020s:** The data engineer changed from building low-level monolithic frameworks to decentralized, modularized, managed and highly abstracted tools. With this greater abstraction and simplification they call the data engineers (data lifecycle engineer) because they are now concerned with security, data management, DataOps, data architecture, orchestration and general data lifecycle management.

### 3. Data Engineering and Data Science?

**A.** Data Engineering is the foundation for every data work because it lay down the sources of the data then ingest what will be used store, transform then serve it to the data science to make ML models or predict outcomes using this data

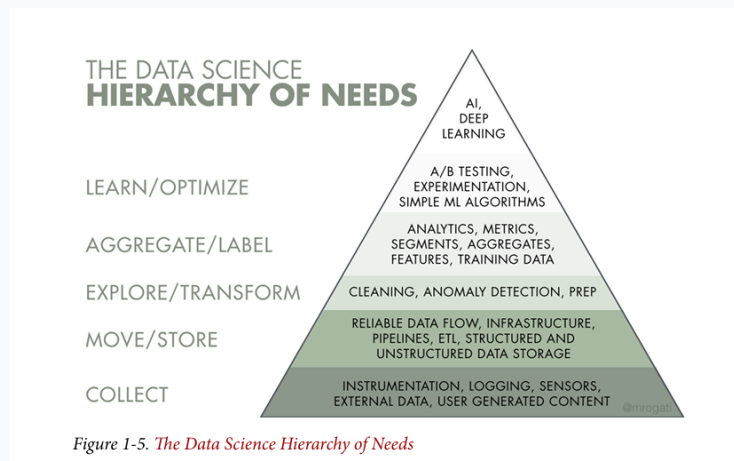


Figure 1-5. The Data Science Hierarchy of Needs

## 4. Data Engineering Skills and Activities?

**A.** The skill of data engineer is the undercurrents of the lifecycle need to understand how all of it fits together in a project and because the simplicity of creating and deploying data engineering projects due to the abstraction of the process, data engineers are now focused on balancing the simplest and most cost-effective, best-of-breed services that deliver value to the business.

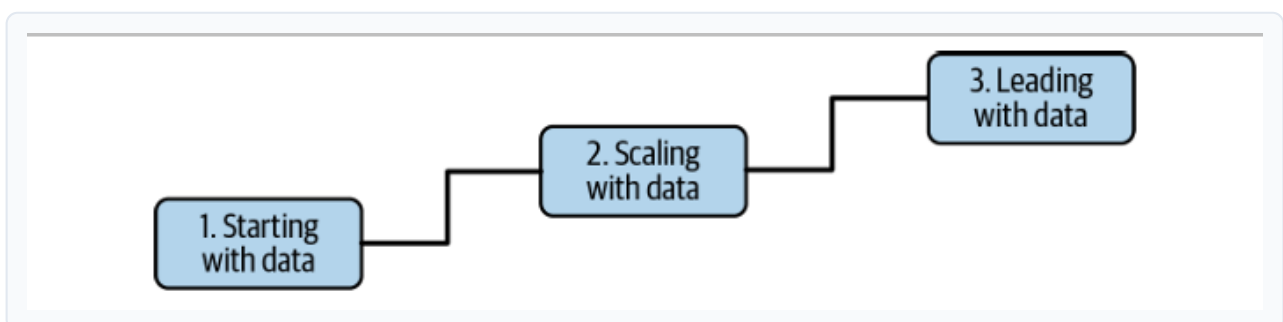
**What are some things a data engineer does not do?** A data engineer typically does not directly build ML models, create reports or dashboards, perform data analysis, build key performance indicators (KPIs), or develop software applications.



## 5. What is Data Maturity?

**A. What is Data Maturity:** It is the progression toward higher data utilization, capabilities, and integration across the organization, in more simple way it is a term used for making the most out of data and know what we can do using this data and how to integrate the data with other companies or anyone needing it.

There is more than Data maturity models but we have this:



- 1. Starting with data:** Building the data foundation before starting in anything else because it will endanger the project to failure if the data architecture and infrastructure wasn't build good enough.

2. **Scaling with data:** With scaling we need to ensure the architecture and infrastructure will handle the high amount of data.
3. **Leading with data:** The automated pipelines and systems created by data engineers allow people within the company to do self-service analytics and ML.

## 6. What is the skills of Data Engineer?

---

### Business Responsibilities:

- Know how to communicate with nontechnical and technical people.
- Understand how to scope and gather business and product requirements.
- Understand the cultural foundations of Agile, DevOps, and DataOps.
- Control costs.
- Learn continuously.

### Technical Responsibilities:

- Generation
- Storage
- Ingestion
- Transformation
- Serving

### The undercurrents of the data engineering lifecycle are the following:

- Security
- Data management
- DataOps
- Data architecture
- Orchestration
- Software engineering

## 7. what is the Types of Data Engineer (roles A, B)?

---

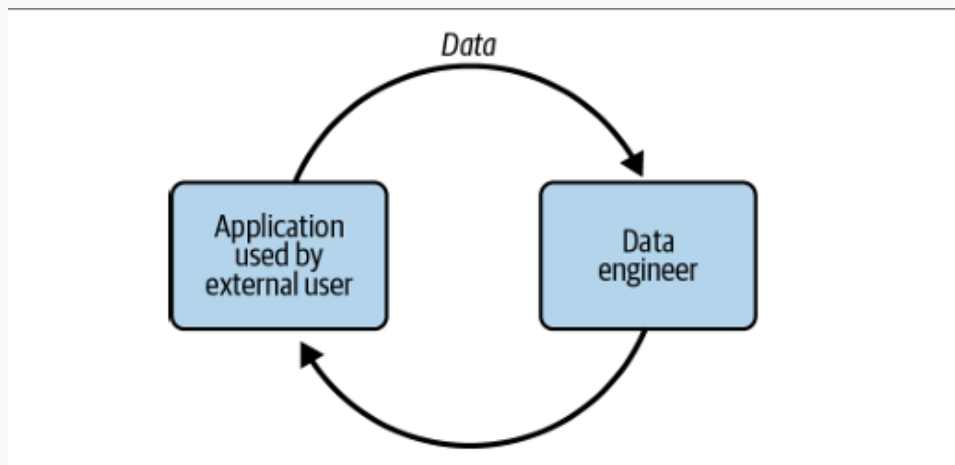
- **Type A data engineers:** The data engineer responsible for keeping data architecture as straightforward as possible and not reinventing the wheel, They manage the lifecycle using existing products and work in a company at all levels of data maturity.
- **Type B data engineers:** B stands for build. they build the data tools and systems that the company need it to scale. They are more found on The 2 and 3 stages of data maturity

## 8. Data Engineers Inside an Organization

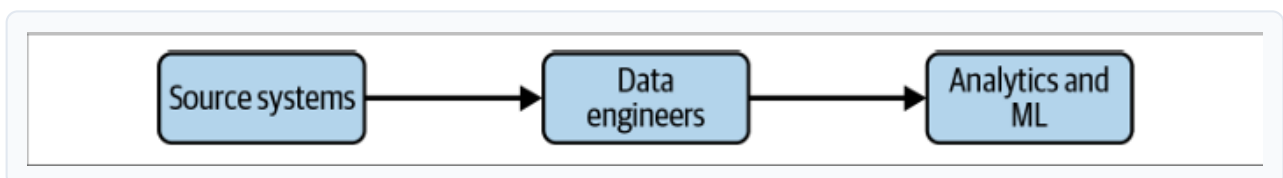
---

### Internal-Facing Versus External-Facing Data Engineers:

**External-Facing:** He aligns with the users of external-facing applications such as social media apps, Internet of Things devices and ecommerce platforms. This data engineer architects, builds, and manages the systems that collect, store, and process transactional and event data from these applications.

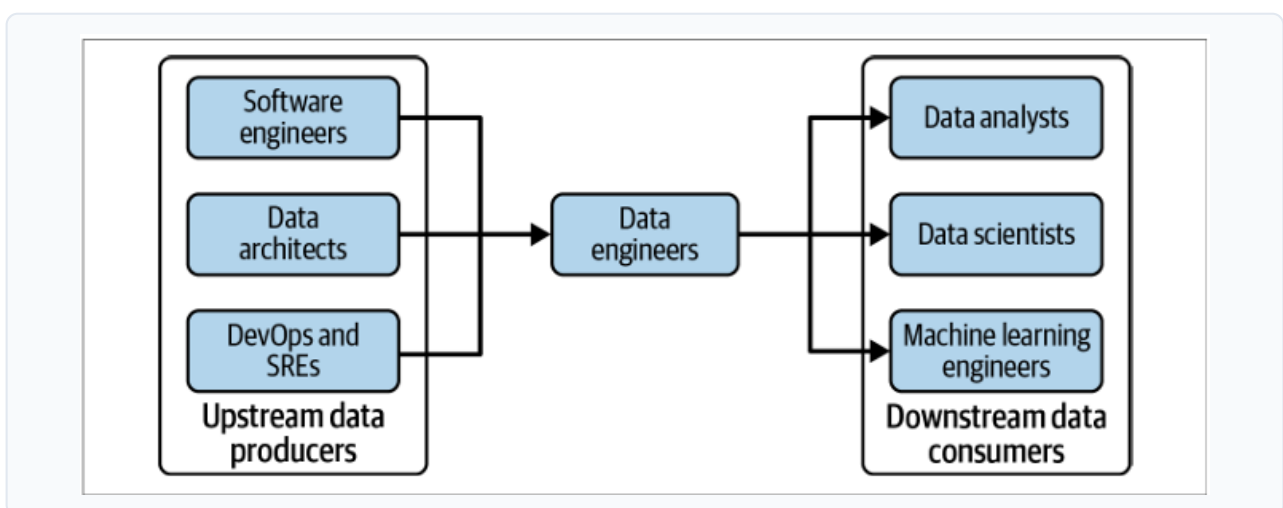


**Internal-Facing:** He focuses on activities crucial to the needs of the business and internal stakeholders like creating and maintaining data pipelines and data warehouses for BI dashboards, reports, business processes, data science and ML models.



## 9. Whom data engineers work with?

**Data Engineers and Other Technical Roles:**



## Chapter 2: The Data Engineering Lifecycle

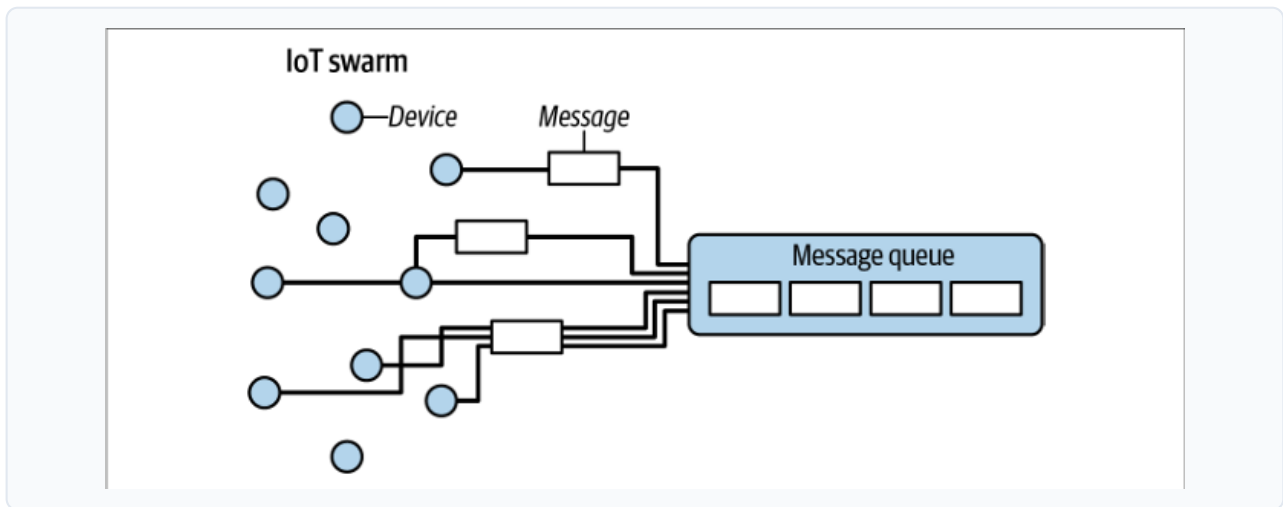
### 1. What Is the Data Engineering Lifecycle?

**A.** It is the process of turning raw data into useful products, ready for consumption by analysts, data scientists, ML engineers and others.

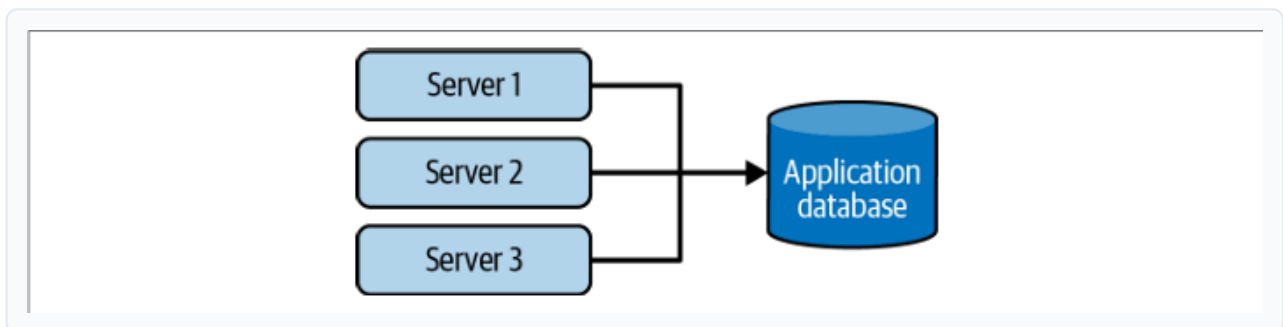
## 2. Generation?

A. The Source System the data engineer consumes data from but doesn't own or control the system itself but he needs to know how it works and generate data and also velocity, variety of the generated data.

- **examples: Traditional source system**



- **IOT Swarm:**



- **Schemaless and fixed schema:**
  - **Schemaless:** is the modern kind of database that doesn't follow the traditional relational database like MongoDB
  - **Fixed-Schema:** is Traditional database

## 3. Storage

A. After we get the data from the source system we need a place to store it, and it could happen in more than one place in the data engineer lifecycle

**Data access frequency: (Hot data, Lukewarm data and Cold data)**

- **Hot data:** data retrieved a lot on the same day\*
- **Lukewarm data:** every week or month
- **Cold data:** archive system retrieved when needed

## 4. Ingestion (Batch vs Streaming) & (Pull vs Push)

A. You need to extract the data you need now this is Ingestion the extraction of the needed data.

## Two major data ingestion concepts:

### 1. Batch versus Streaming:

- **Batch:** when large amount of data processed in chunks
- **Streaming:** data processed in real-time while loading to the system

### 2. Pull versus Push :

- **Push:** The source system writes data out to a target (ELT)
- **Pull:** Data is retrieved from the source system (ETL)

## 5. Transformation

---

**A.** Transformation of the data means the data need to change from its original state into something useful for downstream use cases (ML, DS, DA, AI)

## 6. Serving data

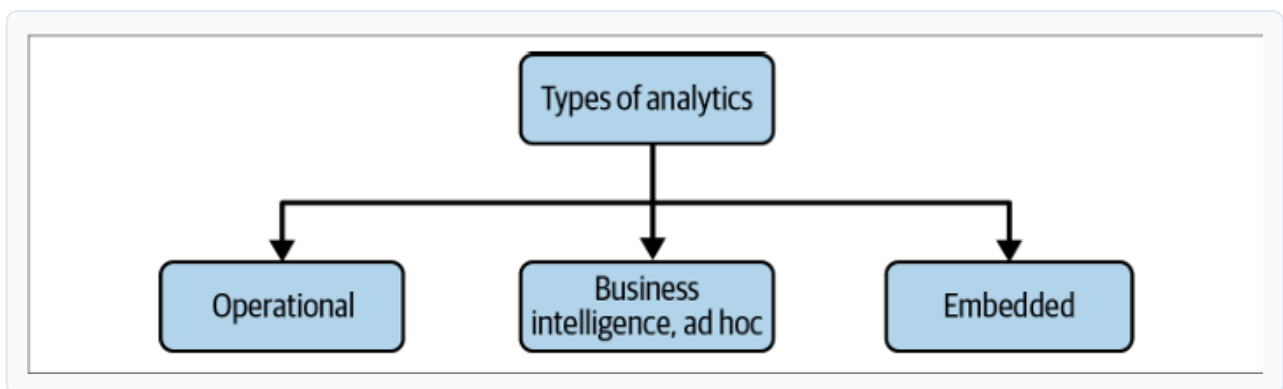
---

**A.** Data serving is the purpose of the whole lifecycle after ingesting, storing and transforming the data its time to get the value out of it

ex, ML engineers can use it to train models, DS can use it to make a prediction model, DA can use it to know what happened

### Analytics:

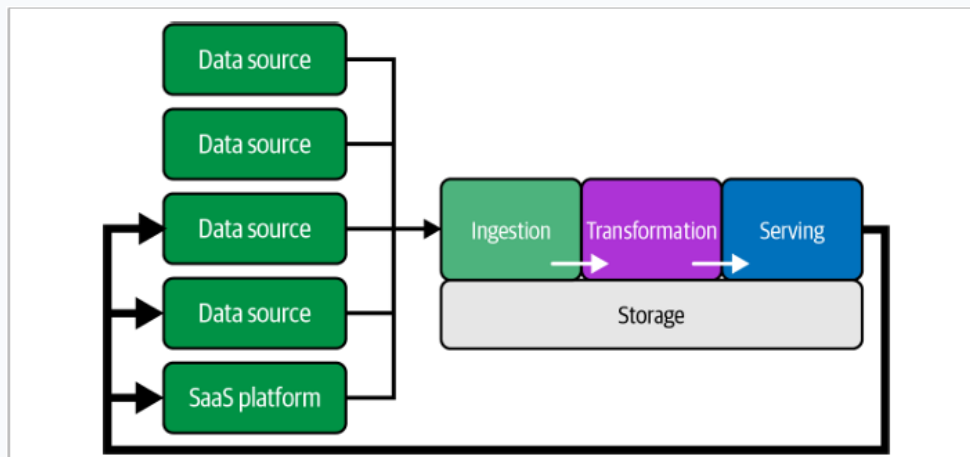
It is the core of the most data adventures after the data engineer lifecycle we need to generate reports or dashboards so we have 3 types of Analytics



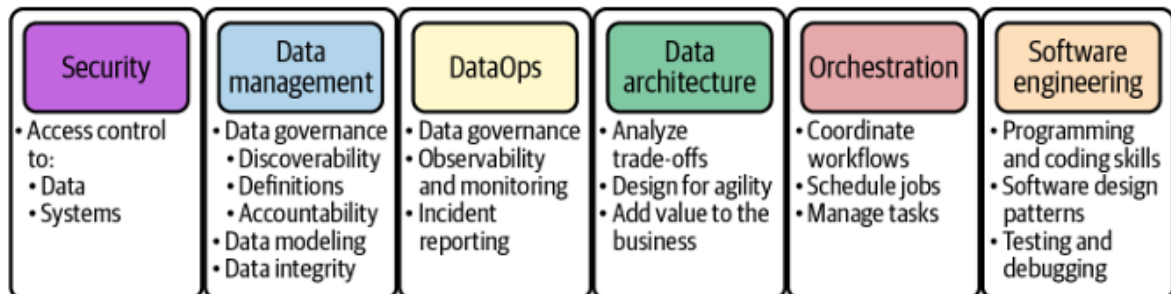
- **BI:** To describe the business past and current state
- **Operational:** Focuses on the details of the current operations without any concern with the past (example: could be a live view of inventory or real-time dashboarding of website or application health)
- **Embedded:** It is the real-time processing for producing reports or dashboards on websites or app (ex: you open your bank account to see what have you spend and on what have you spend it, when and where the site could make charts and dashboards for you and this is embedded analytics)

### Reverse ETL:

After the processed data from the lifecycle end feed it to the source system again



## Data Engineering Lifecycle: Undercurrents



## 6. Security

A. The principle of least privilege means giving a user or system access to only the essential data and resources to perform an intended function.

## 7. Data Management

A. The Data Management Association International (DAMA) Data Management Body of Knowledge (DMBOK), gives definition for data management:

*Data management is the development, execution, and supervision of plans, policies, programs, and practices that deliver, control, protect, and enhance the value of data and information assets throughout their lifecycle.*

In summary data engineers manage the lifecycle and data management set the best practices so data engineers can finish this task.

### Data governance, including discoverability and accountability

According to Data Governance: The Definitive Guide,

*"Data governance is, first and foremost, a data management function to ensure the quality, integrity, security, and usability of the data collected by an organization."*

There is also this expanded definition:

*data governance engages people, processes, and technologies to maximize data value across an organization while protecting data with appropriate security controls.*

**Discoverability,** In a data-driven company, data must be available and discoverable. End users should have quick and reliable access to the data they need to do their jobs. They should know where the data comes from, how it relates to other data, and what the data means.

**Metadata.** Metadata is "data about data," Metadata is exactly the data needed to make data discoverable and governable.

DMBOK identifies four main categories of metadata that are useful to data engineers:

- Business metadata
- Technical metadata
- Operational metadata
- Reference metadata

**Business metadata:** relates to the way data is used in the business, including business and data definitions, data rules and logic, how and where data is used, and the data owner(s).

**Technical metadata:** describes the data created and used by systems across the data engineering lifecycle. It includes the data model and schema, data lineage, field mappings, and pipeline workflows. A data engineer uses technical metadata to create, connect, and monitor various systems across the data engineering lifecycle.

**technical metadata that a data engineer will use:**

- **Pipeline metadata (often produced in orchestration systems):** Pipeline metadata captured in orchestration systems provides details of the workflow schedule, system and data dependencies, configurations, connection details.
- **Data lineage:** Data-lineage metadata tracks the origin and changes to data, and its dependencies, over time.
- **Schema:** Schema metadata describes the structure of data stored in a system such as a database, a data warehouse, a data lake, or a filesystem.
- There is other technical metal data

### **Data accountability.**

Data accountability means assigning an individual to govern a portion of data

why is it important? because Managing data quality is tough if no one is accountable for the data in question.

According to Data Governance: The Definitive Guide, data quality is defined by three main characteristics:

- **Accuracy:** Is the collected data factually correct? Are there duplicate values? Are the numeric values accurate?
- **Completeness:** Are the records complete? Do all required fields contain valid values?
- **Timeliness:** Are records available in a timely fashion?

**Data modeling and design:** It is the process of converting data into a usable form.

**Data lineage:** Describes the recording of data through its lifecycle, tracking both systems that process the data and the upstream data it depends on.

**Data integration and interoperability:** It is the process of integrating data across tools and processes.



**Data lifecycle management:** It is how the data is managed after the lifecycle because of the big amount of data on the cloud and the cost of it and data retrieval  
second problem is the privacy and data retention laws

**Ethics and privacy:** Data now days in impacting people so there should be law and people to protect the data from miss use

## 8. DataOps

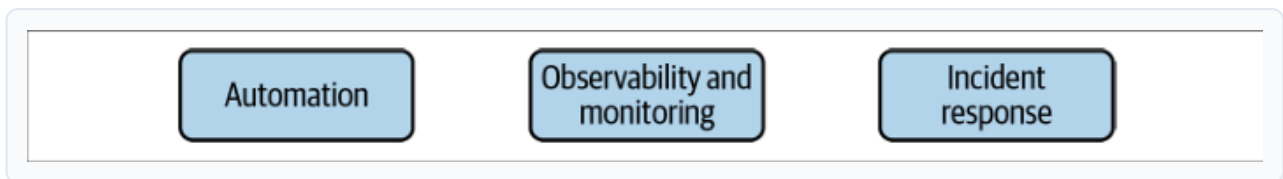
It is a combination of Agile methodology, DevOps and statistical process control to data (DevOps: a job to improve the release and quality of software products, DataOps is the same but for data products)

Data Kitchen (experts in DataOps) describes it :

*DataOps is a collection of technical practices, workflows, cultural norms, and architectural patterns that enable:*

- *Rapid innovation and experimentation delivering new insights to customers with increasing velocity*
- *Extremely high data quality and very low error rates*
- *Collaboration across complex arrays of people, technology, and environments*
- *Clear measurement, monitoring, and transparency of results*

**The three pillars of DataOps:**



- **Automation:** Automation enables reliability and consistency in the DataOps process and allows data engineers to quickly deploy new product features and improvements to existing workflows.
- **Observability and monitoring (Petrella's DODD method):** The purpose of DODD is to give everyone involved in the data chain visibility into the data and data applications so that everyone involved in the data value chain has the ability to identify changes to the data or data applications at every step-from ingestion to transformation to analysis-to help troubleshoot or prevent data issues. DODD focuses on making data observability a first-class consideration in the data engineering lifecycle
- **Incident response:** Incident response is about using the automation and observability capabilities mentioned previously to rapidly identify root causes of an incident and resolve it as reliably and quickly as possible.

## 9. Data architecture

A data architecture reflects the current and future state of data systems that support an organization's long-term data needs and strategy

## 10. Orchestration

Orchestration is the process of coordinating many jobs to run as quickly and efficiently as possible on a scheduled cadence.

## 11. Software engineering

It was very important skill for data engineers before the frameworks that abstracted low-level details

### software engineering skills for data engineering:

- Core data processing code: still important for ingestion, transformation and data serving.
- Development of open source frameworks, like Apache Airflow that still used till now
- Streaming
- Infrastructure as a code (IaC)
- Pipeline as code
- General purpose problem solving

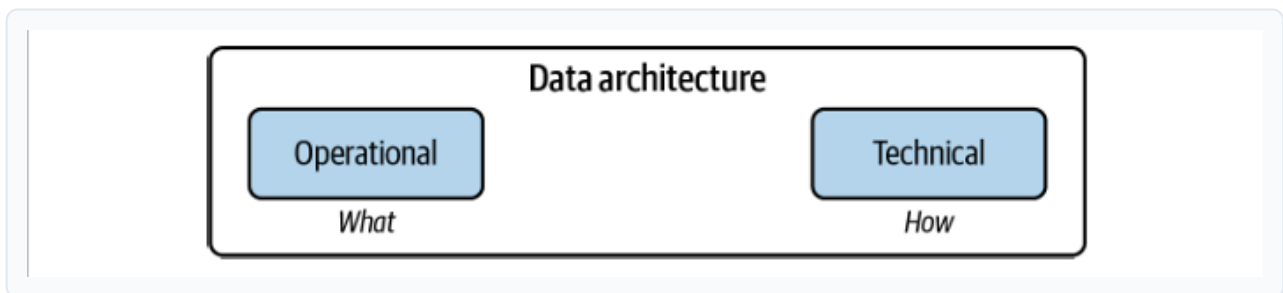
## Chapter 3: Designing Good Data Architecture

### 1. What Is Data Architecture?

**A.** Data architecture is the design of systems to support the evolving data needs of an enterprise, achieved by flexible and reversible decisions reached through a careful evaluation of trade-offs.

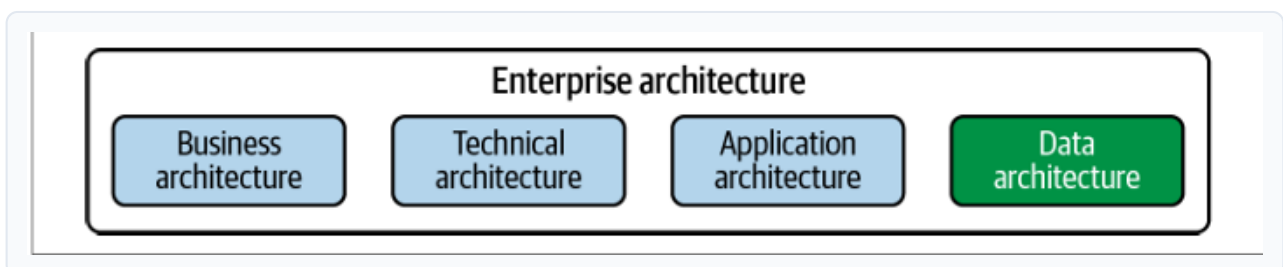
#### Aspects of data architecture (Operational and Technical):

- **Operational architecture:** encompasses the functional requirements of what needs to happen related to people, processes, and technology.
- **Technical architecture:** outlines how data is ingested, stored, transformed, and served along the data engineering lifecycle.



### What is Enterprise architecture?

**A.** Enterprise architecture is the design of systems to support change in the enterprise, achieved by flexible and reversible decisions reached through careful evaluation of trade-offs.



*Never shoot for the best architecture, but rather the least worst architecture.*

**-Mark Richards and Neal Ford**

## 2. Principles of Good Architecture

### 1. Choose common components wisely.

Common components include object storage, version-control systems, observability, monitoring and orchestration systems, and processing engines. Common components should be accessible to everyone with an appropriate use case, and teams are encouraged to rely on common components already in use rather than reinventing the wheel.

### 2. plan for failure

*Everything fails, all the time.*

**-Werner Vogels, CTO of Amazon Web Services**

few terms to evaluate failure scenarios:

- **Availability:** The percentage of time an IT service or component is in an operable state.
- **Reliability:** The system's probability of meeting defined standards in performing its intended function during a specified interval.
- **Recovery time objective:** The maximum acceptable time for a service or system outage. The recovery time objective (RTO) is generally set by determining the business impact of an outage. An RTO of one day might be fine for an internal reporting system. A website outage of just five minutes could have a significant adverse business impact on an online retailer.
- **Recovery point objective:** The acceptable state after recovery. In data systems, data is often lost during an outage. In this setting, the recovery point objective (RPO) refers to the maximum acceptable data loss.

### 3. Architect for scalability.

Scalability in data systems encompasses two main capabilities. First, scalable systems can scale up to handle significant quantities of data.

An elastic system can scale dynamically in response to load, ideally in an automated fashion.

### 4. Architecture is leadership.

**Dave Rice:**

*In many ways, the most important activity of Architectus Oryzus is to mentor the development team, to raise their level so they can take on more complex issues. Improving the development team's ability gives an architect much greater leverage than being the sole decision-maker and thus running the risk of being an architectural bottleneck.*

### 5. Always be architecting.

Data architects don't serve in their role simply to maintain the existing state; instead, they constantly design new and exciting things in response to changes in business and technology.

the EABOK, (Enterprise Architecture Body of Knowledge): an architect's job is to develop deep knowledge of the baseline architecture (current state), develop a target architecture, and map out a sequencing plan to determine priorities and the order of architecture changes.

### 6. Build loosely coupled systems.

*When the architecture of the system is designed to enable teams to test, deploy, and change systems without dependencies on other teams, teams require little communication to get work done. In other words, both the architecture and the teams are loosely coupled.*

**-Google DevOps tech architecture guide**

1. Many small teams engineer a large, complex system. Each team is tasked with engineering, maintaining, and improving some system components.
2. These teams publish the abstract details of their components to other teams via API definitions, message schemas, etc. Teams need not concern themselves with other teams' components; they simply use the published API or message specifications to call these components. They iterate their part to improve their performance and capabilities over time. They might also publish new capabilities as they are added or request new stuff from other teams. Again, the latter happens without teams needing to worry about the internal technical details of the requested features. Teams work together through loosely coupled communication.
3. As a consequence of characteristic 2, each team can rapidly evolve and improve its component independently of the work of other teams.
4. Specifically, characteristic 3 implies that teams can release updates to their components with minimal downtime. Teams release continuously during regular working hours to make code changes and test them

#### **7. Make reversible decisions.**

Bezos said "If you walk through and don't like what you see on the other side, you can't get back to before. We can call these Type 1 decisions. But most decisions aren't like that—they are changeable, reversible—they're two-way doors." Aim for two-way doors whenever possible.

#### **8. Prioritize security.**

Two main ideas: zero-trust security and the shared responsibility security model.

##### **Hardened-perimeter and zero-trust security models:**

It is a security model that treats all the users and devices as untrusted, regardless of their location or identity. Every user, device, and network flow is authenticated and authorized before access is granted.

*resources: Perimeter Security vs Zero Trust: Paving the Way for Cybersecurity. Transformation Tufin and Zero Trust vs Traditional Perimeter Security: Which Security Model Is Better?*

##### **The shared responsibility model:**

Amazon emphasizes the shared responsibility model, which divides security into the security of the cloud and security in the cloud. AWS is responsible for the security of the cloud:

*AWS is responsible for protecting the infrastructure that runs AWS services in the AWS Cloud. AWS also provides you with services that you can use securely.*

##### **AWS users are responsible for security in the cloud:**

*Your responsibility is determined by the AWS service that you use. You are also responsible for other factors including the sensitivity of your data, your organization's requirements, and applicable laws and regulations.*

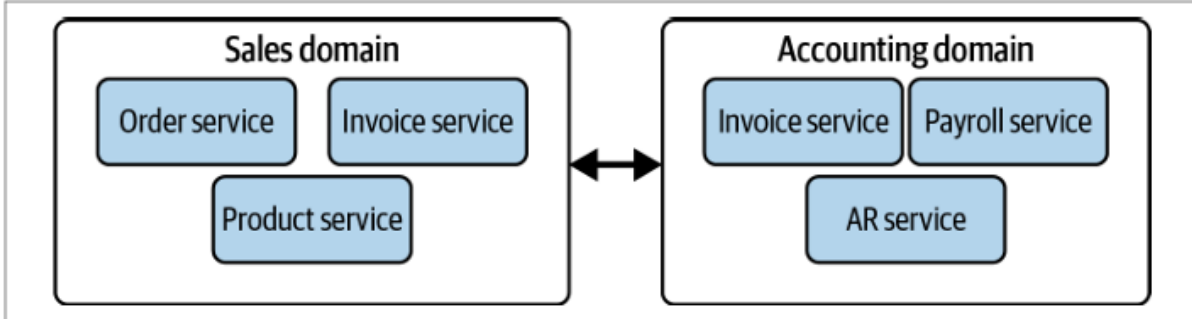
#### **9. Embrace FinOps.**

FinOps is an evolving cloud financial management discipline and cultural practice that enables organizations to get maximum business value by helping engineering, finance, technology, and business teams to collaborate on data-driven spending decisions.

### 3. Major Architecture Concepts

#### Domain and Services

A domain is the real-world subject area for which you're architecting. A service is a set of functionality whose goal is to accomplish a task.

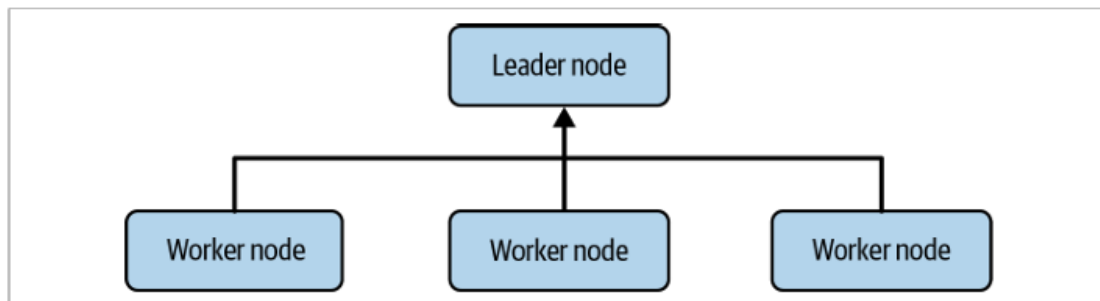


*Figure 3-3. Two domains (sales and accounting) share a common service (invoices), and sales and accounting own their respective domains*

Figure 3-3. Two domains (sales and accounting) share a common service (invoices), and sales and accounting own their respective domains

#### Distributed Systems, Scalability, and Designing for Failure

- **Scalability:** Allows us to increase the capacity of a system to improve performance and handle the demand. For example, we might want to scale a system to handle a high rate of queries or process a huge data set.
- **Elasticity:** The ability of a scalable system to scale dynamically; a highly elastic system can automatically scale up and down based on the current workload. Scaling up is critical as demand increases, while scaling down saves money in a cloud environment. Modern systems sometimes scale to zero, meaning they can automatically shut down when idle.
- **Availability:** The percentage of time an IT service or component is in an operable state.
- **Reliability:** The system's probability of meeting defined standards in performing its intended function during a specified interval.



*Figure 3-4. A simple horizontal distributed system utilizing a leader-follower architecture, with one leader node and three worker nodes*

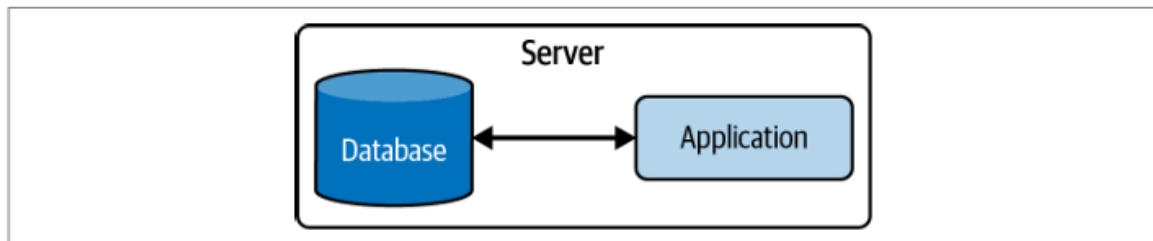
Figure 3-4. A simple horizontal distributed system utilizing a leader-follower architecture, with one leader node and three worker nodes

## Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices

### Single tier.

In a single-tier architecture, your database and application are tightly coupled, residing on a single server. This server could be your laptop or a single virtual machine (VM) in the cloud.

While single-tier architectures are good for prototyping and development, they are not advised for production environments because of the obvious failure risks.

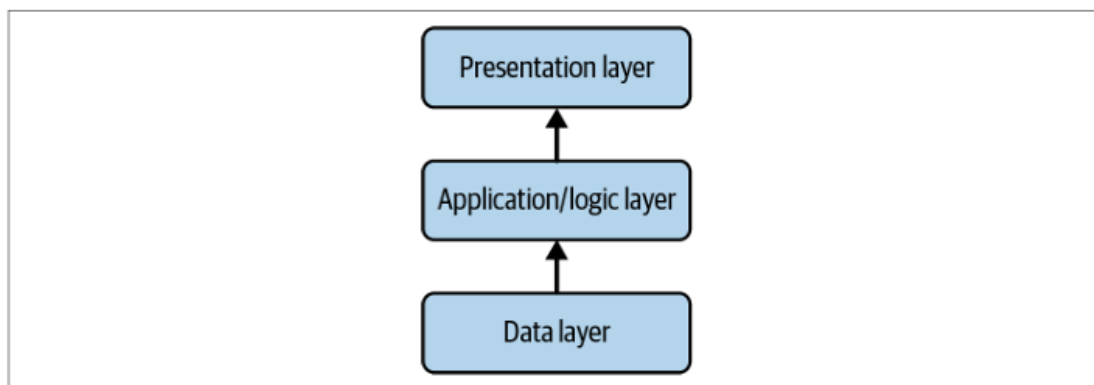


*Figure 3-5. Single-tier architecture*

*Figure 3-5. Single-tier architecture*

### Multitier.

A multitier (also known as n-tier) architecture is composed of separate layers: data, application, business logic, presentation, etc. These layers are bottom-up and hierarchical, meaning the lower layer isn't necessarily dependent on the upper layers; the upper layers depend on the lower layers. The notion is to separate data from the application, and application from the presentation.



*Figure 3-6. A three-tier architecture*

*Figure 3-6. A three-tier architecture*

### Monoliths.

A monolithic application is a software system where all functionalities exist within a single codebase. It is built as one large, unified block, with tightly integrated components that are developed and deployed together.

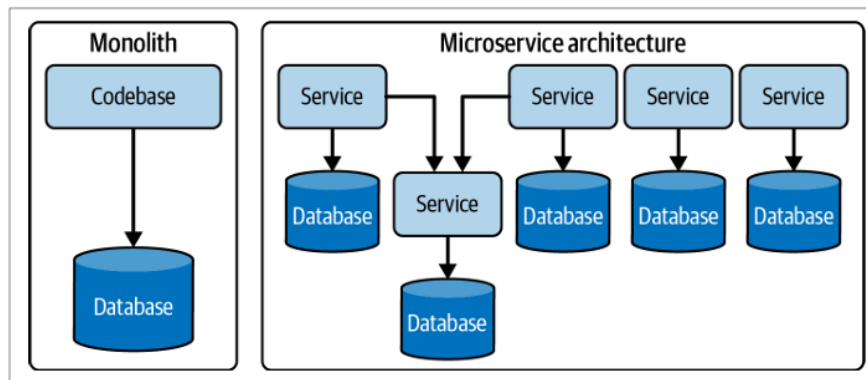
- Single Deployment: All features are packaged and deployed as one unit.
- Tightly Coupled Components: Changes in one part can affect the entire system.

*resources: Analysis of Monolithic and Distributed Systems - Learn System Design - GeeksforGeeks*

### Microservices.

Microservices architecture comprises separate, decentralized, and loosely coupled services. Each service has a

specific function and is decoupled from other services operating within its domain. If one service temporarily goes down, it won't affect the ability of other services to continue functioning.



*Figure 3-7. An extremely monolithic architecture runs all functionality inside a single codebase, potentially colocating a database on the same host server*

*Figure 3-7. An extremely monolithic architecture runs all functionality inside a single codebase, potentially colocating a database on the same host server*

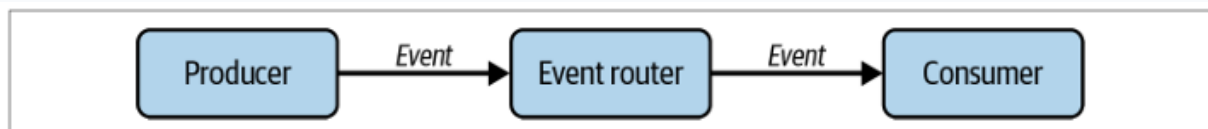
### User Access: Single Versus Multitenant

As a data engineer, you have to make decisions about sharing systems across multiple teams, organizations, and customers.

We have two factors to consider in multitenancy: performance and security.

### Event-Driven Architecture

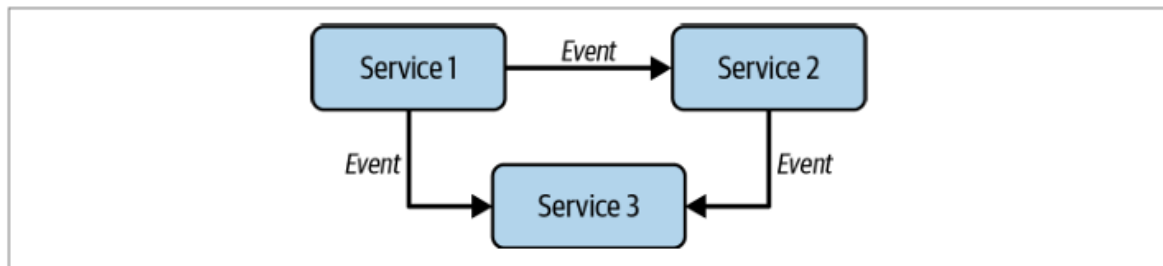
An event-driven workflow encompasses the ability to create, update, and asynchronously move events across various parts of the data engineering lifecycle.



*Figure 3-8. In an event-driven workflow, an event is produced, routed, and then consumed*

*Figure 3-8. In an event-driven workflow, an event is produced, routed, and then consumed*

The advantage of an event-driven architecture is that it distributes the state of an event across multiple services.



*Figure 3-9. In an event-driven architecture, events are passed between loosely coupled services*

*Figure 3-9. In an event-driven architecture, events are passed between loosely coupled services*

## Brownfield Versus Greenfield Projects

**Brownfield projects** often involve refactoring and reorganizing an existing architecture and are constrained by the choices of the present and past. Because a key part of architecture is change management, you must figure out a way around these limitations and design a path forward to achieve your new business and technical objectives.

A **greenfield project** allows you to pioneer a fresh start, unconstrained by the history or legacy of a prior architecture. Greenfield projects tend to be easier than brownfield projects, and many data architects and engineers find them more fun! You have the opportunity to try the newest and coolest tools and architectural patterns. What could be more exciting?

## 3. Examples and Types of Data Architecture

### Data Warehouse

A data warehouse is a central data hub used for reporting and analysis. Data in a data warehouse is typically highly formatted and structured for analytics use cases. It's among the oldest and most well-established data architectures.

In 1989, Bill Inmon originated the notion of the data warehouse, which he described as "*a subject-oriented, integrated, nonvolatile, and time-variant collection of data in support of management's decisions.*" Though technical aspects of the data warehouse have evolved significantly, we feel this original definition still holds its weight today.

There are two types of data warehouse architecture: organizational and technical.

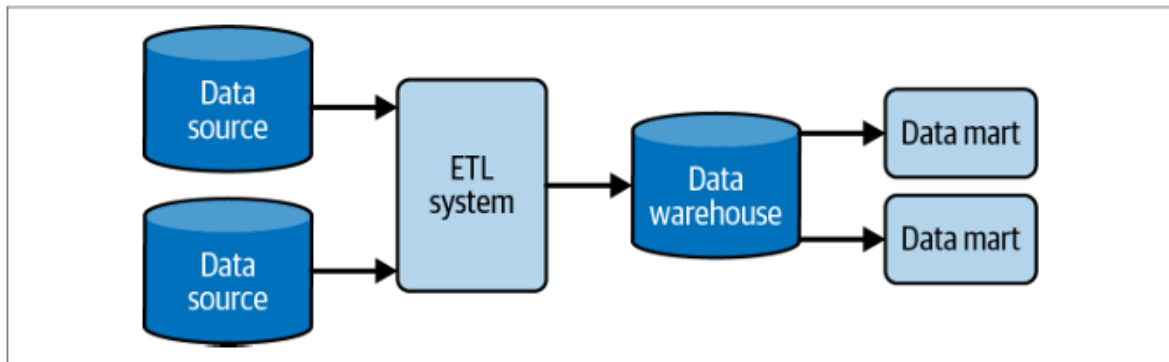
The organizational data warehouse architecture organizes data associated with certain business team structures and processes.

The technical data warehouse architecture reflects the technical nature of the data warehouse, such as MPP.

The organizational data warehouse architecture has two main characteristics:

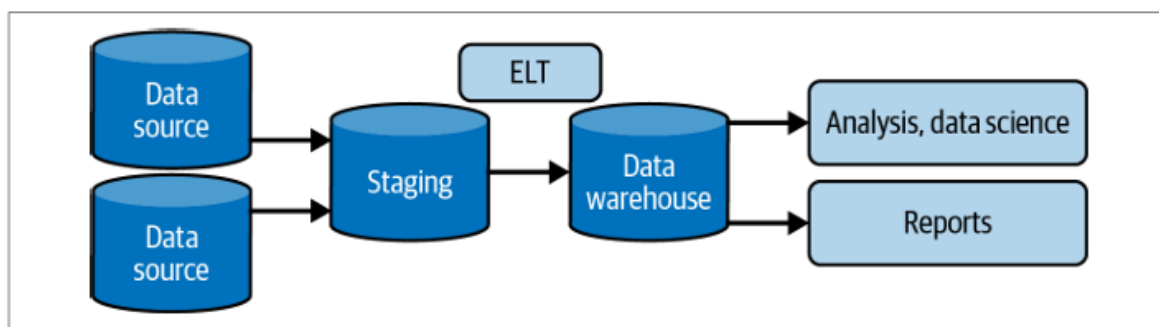
1. Separates online analytical processing (OLAP) from production databases (online transaction processing)
2. Centralizes and organizes data





*Figure 3-10. Basic data warehouse with ETL*

*Figure 3-10. Basic data warehouse with ETL*



*Figure 3-11. ELT—extract, load, and transform*

*Figure 3-11. ELT-extract, load, and transform*

### The cloud data warehouse

Cloud data warehouses represent a significant evolution of the on-premises data warehouse architecture and have thus led to significant changes to the organizational architecture.

### Data marts

A data mart is a more refined subset of a warehouse designed to serve analytics and reporting, focused on a single suborganization, department, or line of business; every department has its own data mart, specific to its needs. This is in contrast to the full data warehouse that serves the broader organization or business.

### Data Lakes

Instead of imposing tight structural limitations on data, why not simply dump all of your data-structured and unstructured-into a central location? The data lake promised to be a democratizing force, liberating the business to drink from a fountain of limitless data. The first-generation data lake, "data lake 1.0," made solid contributions but generally failed to deliver on its promise.

Many organizations found significant value in data lakes-especially huge, heavily data-focused Silicon Valley tech companies like Netflix and Facebook. These companies had the resources to build successful data practices and create their custom Hadoop-based tools and enhancements. But for many organizations, data lakes turned into an internal superfund site of waste, disappointment, and spiraling costs.

### Convergence, Next-Generation Data Lakes, and the Data Platform

Databricks introduced the notion of a data lakehouse. The lakehouse incorporates the controls, data management, and data structures found in a data warehouse while still housing data in object storage and

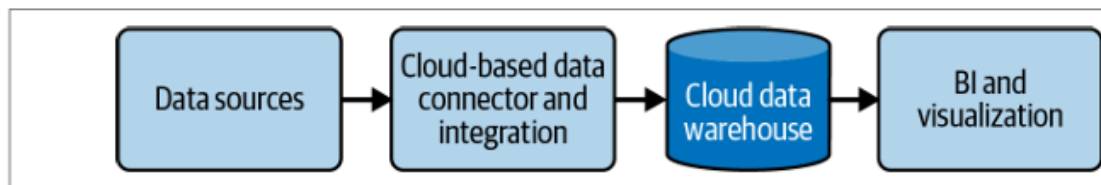
supporting a variety of query and transformation engines.

The term data lakehouse suggests a convergence between data lakes and data warehouses.

We now see several vendors offering data platforms that combine data lake and data warehouse capabilities. From our perspective, AWS, Azure, Google Cloud, Snowflake, and Databricks

### Modern Data Stack

Whereas past data stacks relied on expensive, monolithic toolsets, the main objective of the modern data stack is to use cloud-based, plug-and-play, easy-to-use, off-the-shelf components to create a modular and cost-effective data architecture. These components include data pipelines, storage, transformation, data management/ governance, monitoring, visualization, and exploration.

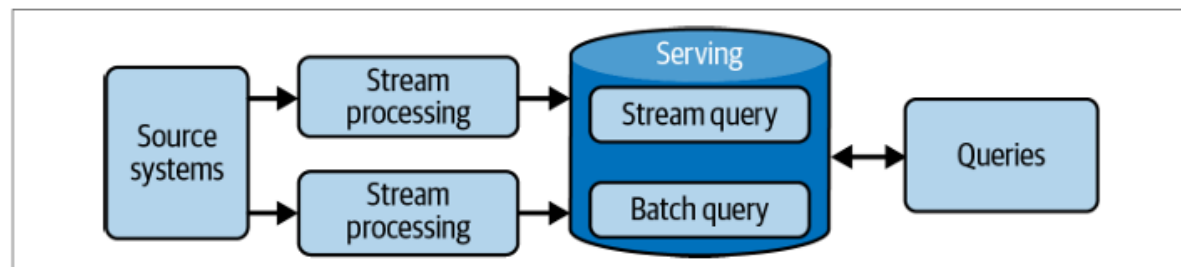


*Figure 3-13. Basic components of the modern data stack*

*Figure 3-13. Basic components of the modern data stack*

### Lambda Architecture

In a Lambda architecture, you have systems operating independently of each other- batch, streaming, and serving. The source system is ideally immutable and append-only, sending data to two destinations for processing: stream and batch.



*Figure 3-14. Lambda architecture*

*Figure 3-14. Lambda architecture*

### Kappa Architecture

As a response to the shortcomings of Lambda architecture, Jay Kreps proposed an alternative called Kappa architecture. The central thesis is this: why not just use a stream-processing platform as the backbone for all data handling-ingestion, storage, and serving? This facilitates a true event-based architecture.

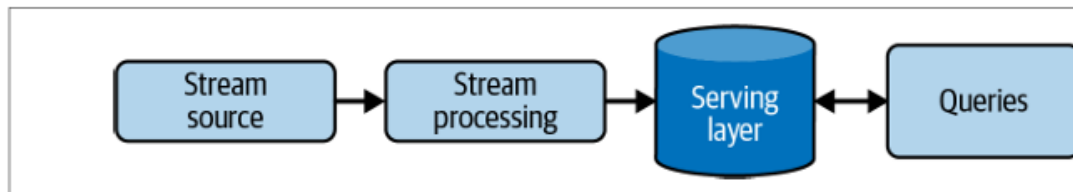


Figure 3-15. Kappa architecture

Figure 3-15. Kappa architecture

It's not widely used because it's hard to implement and practice

### The Dataflow Model and Unified Batch and Streaming

The core idea in the Dataflow model is to view all data as events, as the aggregation is performed over various types of windows. Ongoing real-time event streams are unbounded data. Data batches are simply bounded event streams, and the boundaries provide a natural window. Engineers can choose from various windows for real-time aggregation, such as sliding or tumbling. Real-time and batch processing happens in the same system using nearly identical code

### Architecture for IoT

```
def Talk_about_iot():
    pass
```

### Data Mesh

The data mesh attempts to invert the challenges of centralized data architecture, taking the concepts of domain-driven design (commonly used in software architectures) and applying them to data architecture. Because the data mesh has captured much recent attention, you should be aware of it.

A big part of the data mesh is decentralization, as Zhamak Dehghani noted in her groundbreaking article on the topic:

*In order to decentralize the monolithic data platform, we need to reverse how we think about data, its locality, and ownership. Instead of flowing the data from domains into a centrally owned data lake or platform, domains need to host and serve their domain datasets in an easily consumable way.*

Dehghani later identified four key components of the data mesh:

- Domain-oriented decentralized data ownership and architecture
- Data as a product
- Self-serve data infrastructure as a platform
- Federated computational governance

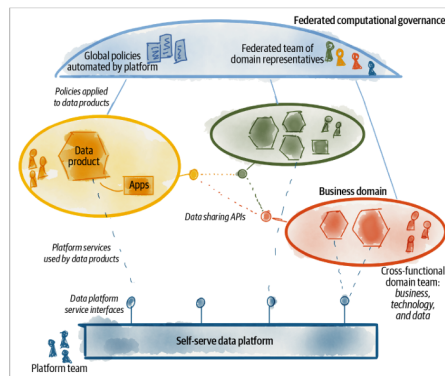


Figure 3-18. Simplified example of a data mesh architecture. Source: From Data Mesh, by Zhamak Dehghani. Copyright © 2022 Zhamak Dehghani. Published by O'Reilly Media, Inc. Used with permission.

Figure 3-18. Simplified example of a data mesh architecture. Source: From Data Mesh, by Zhamak Dehghani. Copyright 2022 Zhamak Dehghani. Published by O'Reilly Media, Inc. Used with permission.

Data architecture have countless variations such as data fabric, data hub, scaled architecture, metadata-first architecture, event-driven architecture, live data stack, and many more.

## Chapter 4: Choosing Technologies Across the Data Engineering Lifecycle

Tactical plan for making technology choices once we have a strategic architecture blueprint.

- Team size and capabilities
- Speed to market
- Interoperability
- Cost optimization and business value
- Today versus the future: immutable versus transitory technologies
- Location (cloud, on prem, hybrid cloud, multi-cloud)
- Build versus buy
- Monolith versus modular
- Serverless versus servers
- Optimization, performance, and the benchmark wars
- The undercurrents of the data engineering lifecycle